

实验二：程序栈帧与内存布局

Part1: 调试 Unix V6++应用程序

一、在 Unix V6++系统中添加应用程序 showStack 并在 Bochs 虚拟机上运行
首先在 src/program/ 下加入 showStack.c 源代码

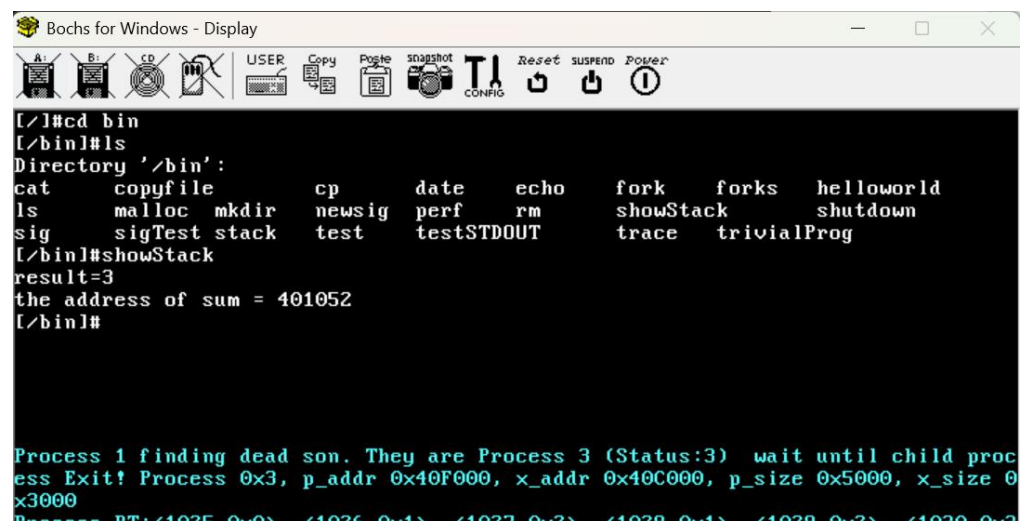
```
#include <stdio.h>
int version=1;

main1()
{
    int a,b,result;
    a = 1;
    b = 2;
    result = sum(a,b);
    printf("result=%d\n",result);
    printf("the address of sum = %0x\n",&sum);
}
int sum(var1, var2)
{
    int count;
    version = 2;
    count = var1 + var2;
    return(count);
}
```

修改 makefile 文件

```
$(TARGET)\showStack.exe \
.....
$(TARGET)\showStack.exe : showStack.c
$(CC) $(CFLAGS) -I"$(INCLUDE)" -I"$(LIB_INCLUDE)" $< -e _main1 $(V6++LIB) -o $@
copy $(TARGET)\showStack.exe $(MAKEIMAGEPATH)\$(BIN)\showStack
```

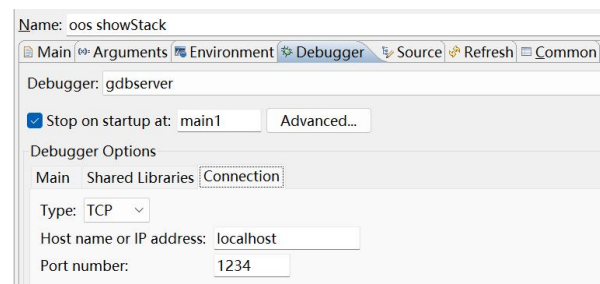
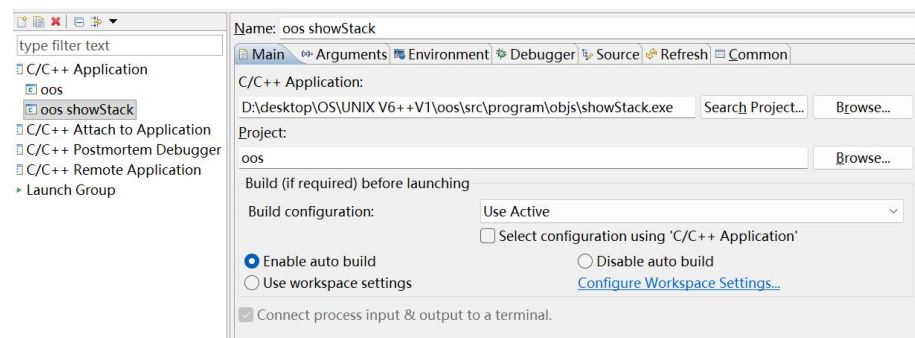
Bochs 虚拟机上直接运行



```
Bochs for Windows - Display
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat      copyfile  cp      date      echo      fork      forks      helloworld
ls        malloc    mkdir   newsig    perf      rm        showStack  shutdown
sig       sigTest   stack   test      testSTDOUT trace     trivialProg
[/bin]#showStack
result=3
the address of sum = 401052
[/bin]#

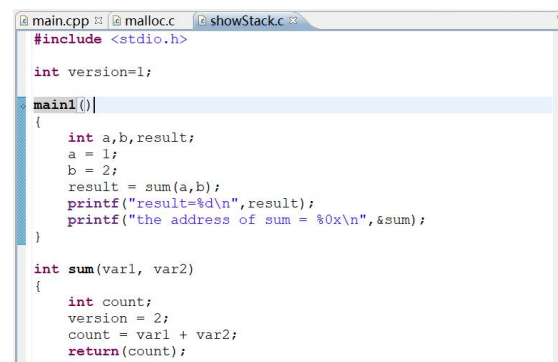
Process 1 finding dead son. They are Process 3 (Status:3) wait until child process Exit! Process 0x3, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0x3000
Process 3 PT: (1025, 0x0) (1026, 0x1) (1027, 0x2) (1028, 0x1) (1028, 0x2) (1029, 0x3)
```

二、创建调试配置

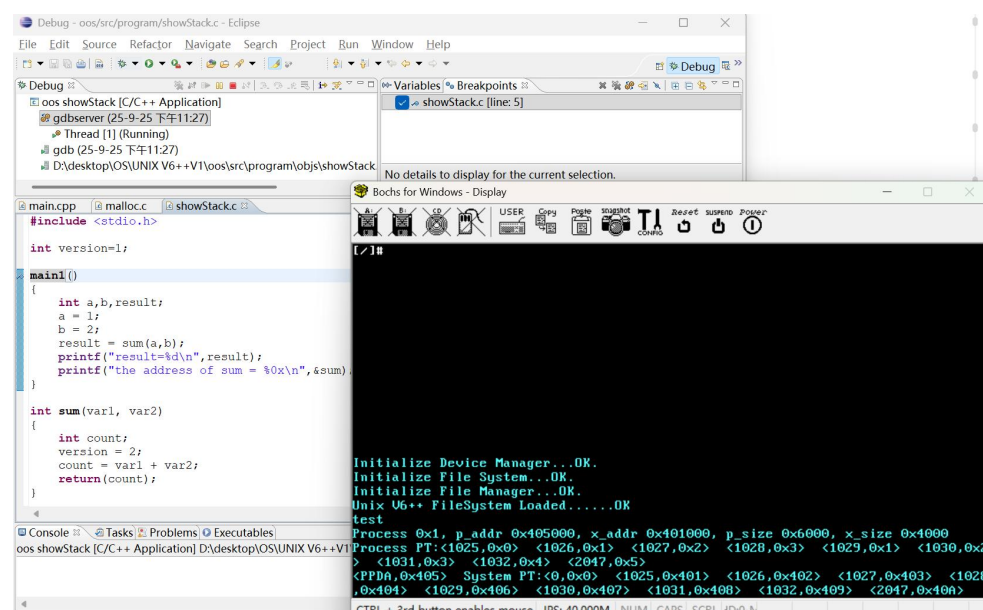


三、调试 showStack.exe, 观察程序运行过程中栈帧的变化, 关注寄存器 EIP、ESP、EBP 和 EAX

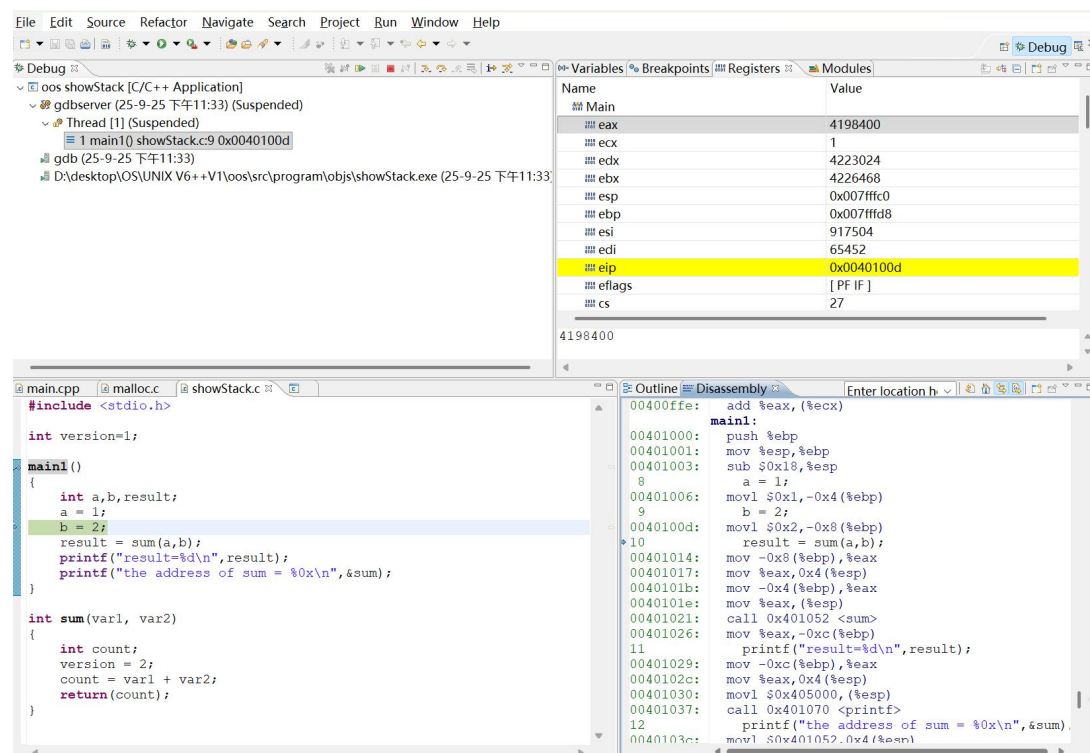
打断点, 这里选择在 main1 函数处打断点



开始调试



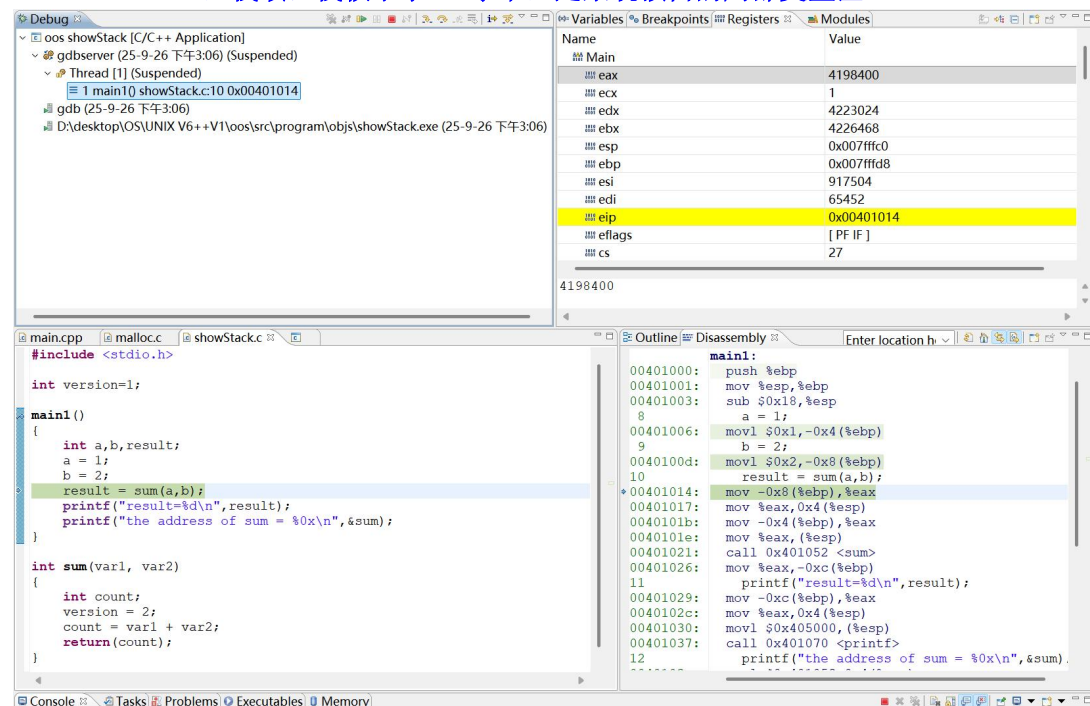
打开 Register 和 Disassembly 视图，可以观察汇编代码和各寄存器的值



执行 sum 函数之前，EIP = 0x00401021。

EBP = 0x007fffd8 也就是 main1 函数栈帧基地址，

ESP = 0x007fffc0，栈顶比栈帧小了 24 字节，是系统预留的局部变量区。

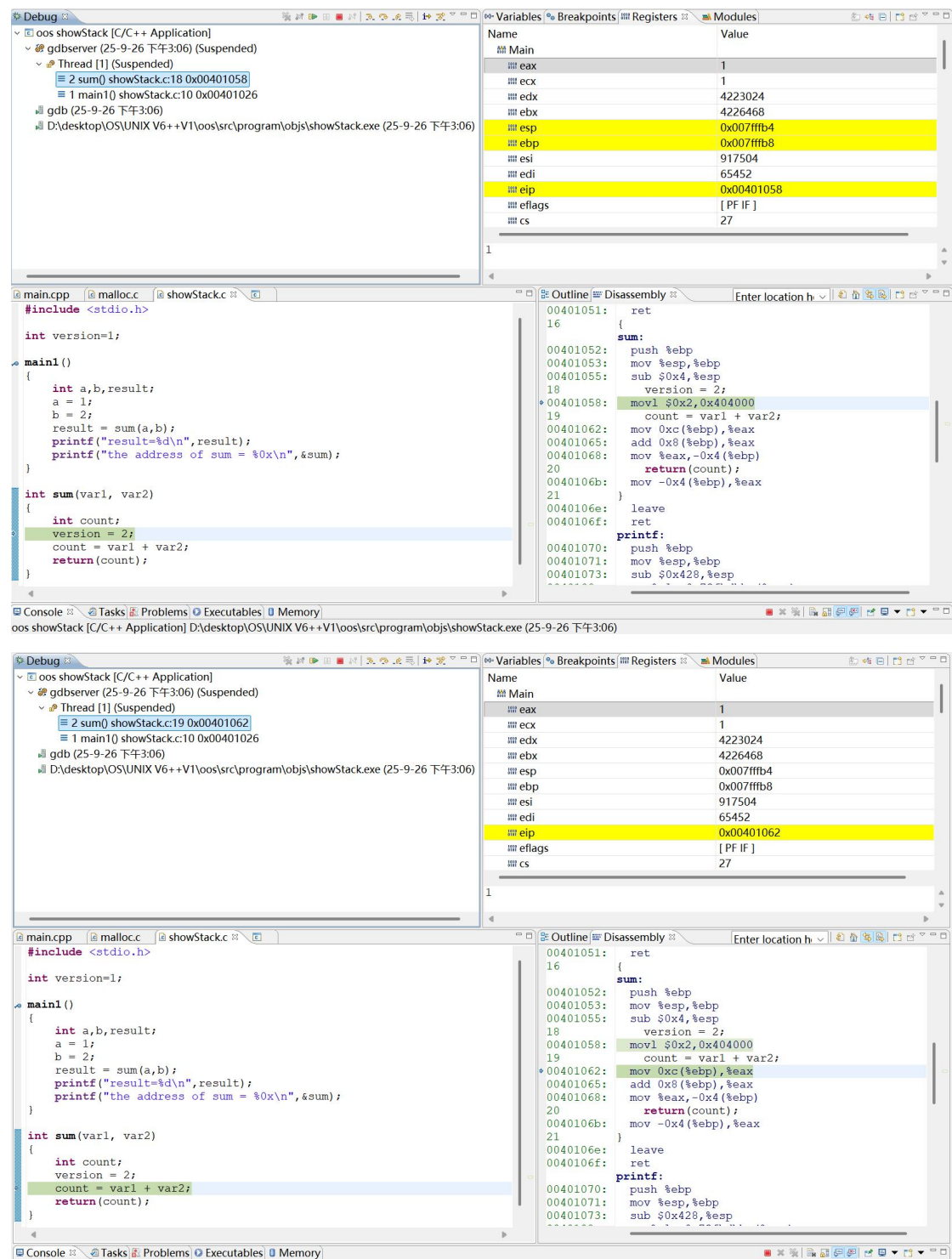


开始执行 sum 函数

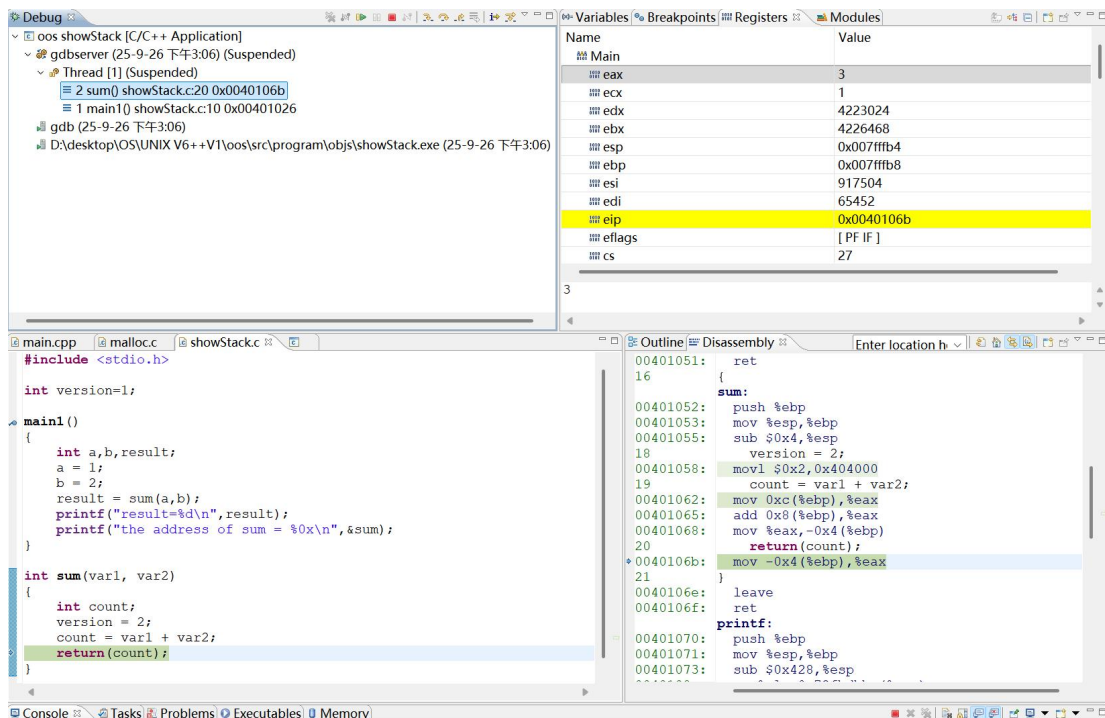
EIP = 0x00401058，跳转到 sum 函数内部

EBP = 0x007fffb8，表示 sum 函数的栈帧基地址

ESP = 0x007fffb4，比 EBP 小了 4，是局部变量 count 的大小



执行完加法操作，EAX 寄存器的值变为 3，存放 a 与 b 的和

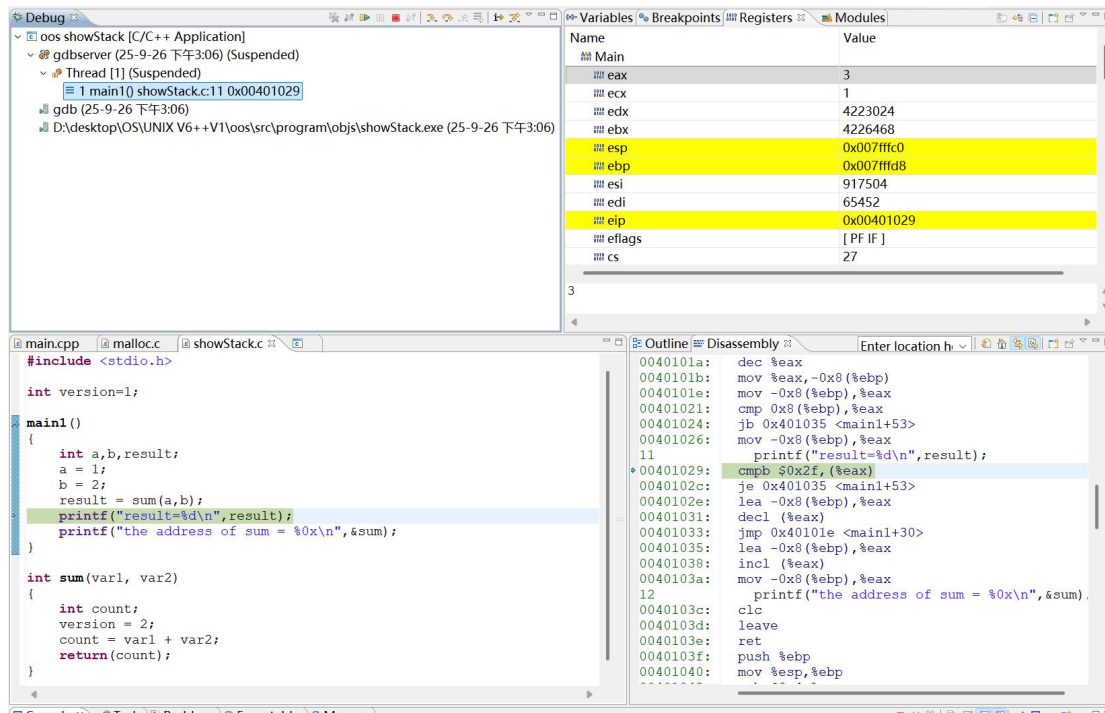


Name	Value
var1	1
var2	2
count	3

返回 main1 执行后续打印步骤

EIP = 0x00401029, 跳转到 main1 函数的返回地址

EBP = 0x007fffd8, ESP = 0x007fffc0, 与原先执行 sum 函数之前相同



Part2: 用工具 objdump 探索 Unix V6++可执行程序

一、查看 section headers

```
D:\desktop\OS\UNIX V6++V1\oos\tools>objdump -h showStack.exe

showStack.exe:      file format pei-i386

Sections:
Idx Name              Size      VMA          LMA          File off  Algn
  0 .text              000020fc  00401000     00401000     00000400  2**2
                        CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .data              0000000c  00404000     00404000     00002600  2**2
                        CONTENTS, ALLOC, LOAD, DATA
  2 .rdata             0000021c  00405000     00405000     00002800  2**3
                        CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .bss               00000020  00406000     00406000     00000000  2**2
                        ALLOC
  4 .idata             00000014  00407000     00407000     00002c00  2**2
                        CONTENTS, ALLOC, LOAD, DATA
  5 .stab              00003ff0  00408000     00408000     00002e00  2**2
                        CONTENTS, READONLY, DEBUGGING, EXCLUDE
  6 .stabstr           0000124d  0040c000     0040c000     00006e00  2**0
                        CONTENTS, READONLY, DEBUGGING, EXCLUDE
```

二、查看可执行码

```
D:\desktop\OS\UNIX V6++V1\oos\tools>objdump -d showStack.exe

showStack.exe:      file format pei-i386

Disassembly of section .text:

00401000 <_main1>:
401000: 55                push    %ebp
401001: 89 e5            mov     %esp,%ebp
401003: 83 ec 18        sub     $0x18,%esp
401006: c7 45 fc 01 00 00 00 movl    $0x1,-0x4(%ebp)
40100d: c7 45 f8 02 00 00 00 movl    $0x2,-0x8(%ebp)
401014: 8b 45 f8        mov     -0x8(%ebp),%eax
401017: 89 44 24 04      mov     %eax,0x4(%esp)
40101b: 8b 45 fc        mov     -0x4(%ebp),%eax
40101e: 89 04 24        mov     %eax,(%esp)
401021: e8 2c 00 00 00  call    401052 <_sum>
401026: 89 45 f4        mov     %eax,-0xc(%ebp)
401029: 8b 45 f4        mov     -0xc(%ebp),%eax
40102c: 89 44 24 04      mov     %eax,0x4(%esp)
401030: c7 04 24 00 50 40 00 movl    $0x405000,(%esp)
401037: e8 34 00 00 00  call    401070 <_printf>
40103c: c7 44 24 04 52 10 40 movl    $0x401052,0x4(%esp)
401043: 00
401044: c7 04 24 0b 50 40 00 movl    $0x40500b,(%esp)
40104b: e8 20 00 00 00  call    401070 <_printf>
401050: c9              leave   %eax
401051: c3              ret

00401052 <_sum>:
401052: 55                push    %ebp
401053: 89 e5            mov     %esp,%ebp
401055: 83 ec 04        sub     $0x4,%esp
401058: c7 05 00 40 40 00 02 movl    $0x2,0x404000
40105f: 00 00 00
401062: 8b 45 0c        mov     0xc(%ebp),%eax
401065: 03 45 08        add     0x8(%ebp),%eax
401068: 89 45 fc        mov     %eax,-0x4(%ebp)
40106b: 8b 45 fc        mov     -0x4(%ebp),%eax
40106e: c9              leave   %eax
40106f: c3              ret
```

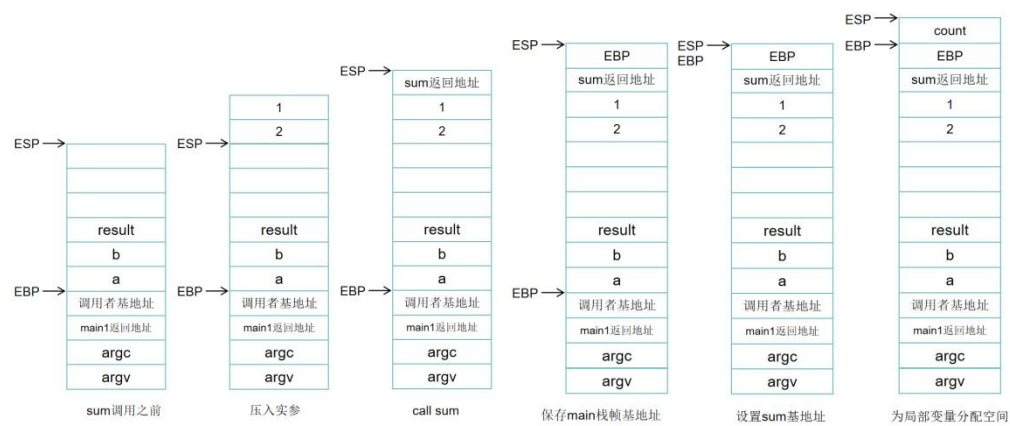
三、回答问题

1.注释 sum()函数汇编码

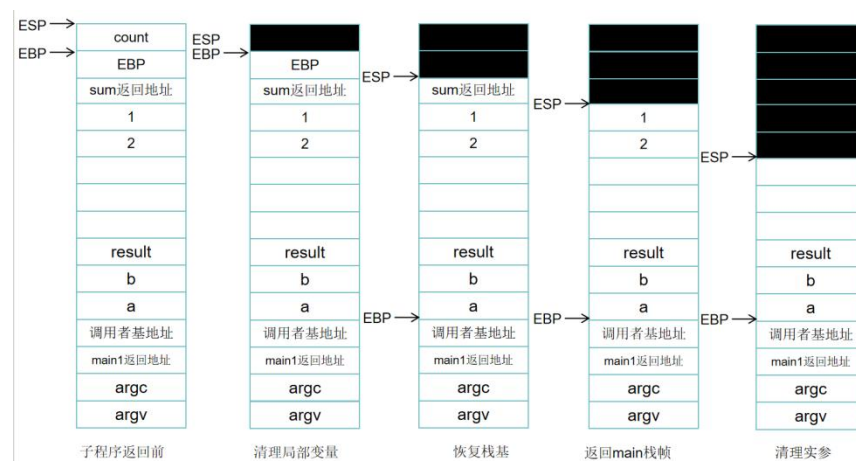
```
00401052 <_sum>:
401052: 55                push    %ebp                // 保存调用者(main1)的栈底指针(ebp)
401053: 89 e5             mov     %esp,%ebp          // 建立sum自己的新栈帧, 将当前栈顶(esp)设为新栈底(ebp)
401055: 83 ec 04         sub     $0x4,%esp          // 在 sum 的栈帧中, 为局部变量区分配4个字节(给count)
401058: c7 05 00 40 00 02 movl    $0x2,0x404000      // version = 2; 这是一个全局变量, 直接通过内存地址(0x404000)赋值
40105f: 00 00 00         mov     0xc(%ebp),%eax     // 读取第2个传入的参数(var2), 其位置在 ebp+12
401062: 8b 45 0c         add     0x8(%ebp),%eax     // 将第1个传入的参数(var1)加到 eax 上, 完成 var1+var2
401065: 03 45 08         mov     %eax,-0x4(%ebp)    // 将计算结果(在eax中)存入局部变量 count (位置在 ebp-4)
401068: 89 45 fc         mov     -0x4(%ebp),%eax    // 将局部变量 count 的值加载到 eax 寄存器, 准备作为返回值
40106b: 8b 45 fc         mov     -0x4(%ebp),%eax    // 清除 sum 函数的局部变量区, 并恢复调用者(main1)的栈帧
40106e: c9              leave   <>
40106f: c3              ret     <>                // 子程序返回, 跳转到 call 指令的下一条指令处
```

2.绘制 showStack 程序的栈。在 debug 视图单步，观察栈帧的创建和销毁过程。绘图展示堆栈的变化过程。

调用子程序：



子程序返回，栈帧销毁：



3.使用 objdump 命令，发现应用程序的更多细节。

使用 objdump -t 命令，可以列出可执行文件的符号表。如图所示，_main1 的值是 0x00000000，_sum 的值是 0x00000052，_printf 的值是 0x00000070。这里的值是相对于其所在段基址的偏移量。因为这些函数都在 .text 段，而从 objdump -h 知道 .text 段的基址 (VMA) 是 0x00401000。所以，要得到它们在内存中的绝对地址，只需要将基址和相对偏移地址相加即可。0x00401000+0x00000052=0x401052，与实际 sum 函数的地址相符合。

```
D:\desktop\OS\UNIX V6++V1\oos\tools>objdump -t showStack.exe

showStack.exe:      file format pei-i386

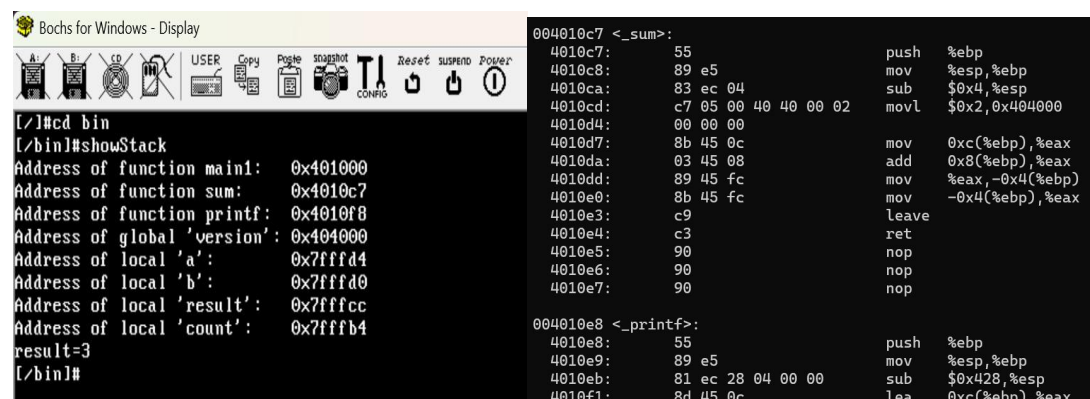
SYMBOL TABLE:
[ 0](sec -2)(fl 0x00)(ty  0)(scl 103) (nx 1) 0x0000000f showStack.c
File
[ 2](sec 1)(fl 0x00)(ty 20)(scl  2) (nx 1) 0x00000000 _main1
AUX tagndx 0 ttlsiz 0x0 lnno 0 next 0
[ 4](sec 1)(fl 0x00)(ty 20)(scl  2) (nx 0) 0x00000052 _sum
[ 5](sec 1)(fl 0x00)(ty  0)(scl  3) (nx 1) 0x00000000 .text
AUX scnlen 0x70 nreloc 6 nlno 0
```

4. (1) 修改 showStack() 程序，printf 出来 main, sum, printf, version 以及所有局部变量的地址，这些地址是否与你期待的相符。

```
main1()
{
    int a,b,result;
    a = 1;
    b = 2;
    printf("Address of function main1:  0x%x\n", &main1);
    printf("Address of function sum:      0x%x\n", &sum);
    printf("Address of function printf:  0x%x\n", &printf);
    printf("Address of global 'version': 0x%x\n", &version);
    printf("Address of local 'a':         0x%x\n", &a);
    printf("Address of local 'b':         0x%x\n", &b);
    printf("Address of local 'result':    0x%x\n", &result);
    result = sum(a,b);
    printf("result=%d\n",result);
}
```

结合 part1 的调试过程中观察到的 ebp 和 esp 的值，这些地址与我期待的无误。

EBP=0x007fffd8，而变量 a 和变量 b，变量 result 的地址分别相差 4，与第 3 问中画的栈结果相同。修改后的可执行文件用 objdump 调试，显示的 main1、sum、printf 函数地址与打印的也一致。（由于 tools 目录下的可执行文件是从 src\program\objs 中复制过来的，所以重新编译后需要将新文件复制过去再使用 objdump 命令，本人因为这个原因浪费了半小时）



Bochs for Windows - Display

Left pane (Command Prompt):

```
[/]\cd bin
[/bin]#showStack
Address of function main1:  0x401000
Address of function sum:    0x4010c7
Address of function printf: 0x4010f8
Address of global 'version': 0x404000
Address of local 'a':       0x7fffd4
Address of local 'b':       0x7fffd0
Address of local 'result':  0x7fffcc
Address of local 'count':   0x7fffb4
result=3
[/bin]#
```

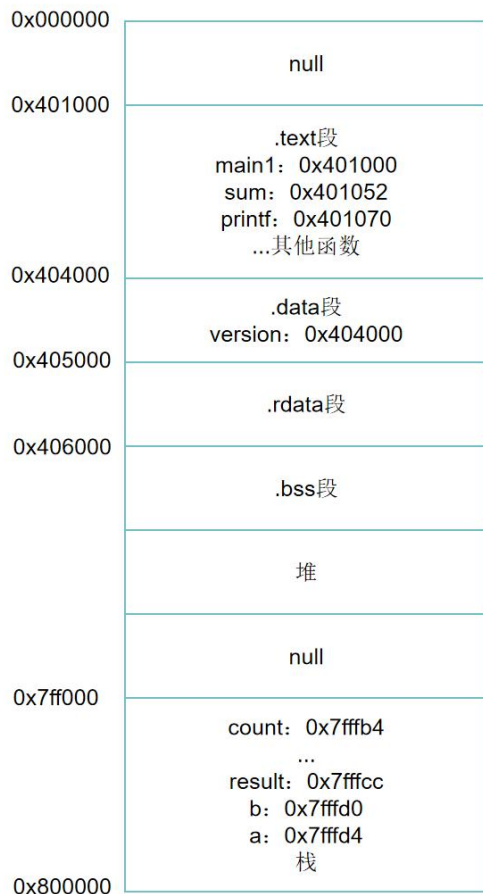
Right pane (Assembly Code):

```
004010c7 <_sum>:
4010c7: 55          push    %ebp
4010c8: 89 e5       mov     %esp,%ebp
4010ca: 83 ec 04    sub     $0x4,%esp
4010cd: c7 05 00 40 00 02 movl    $0x2,0x404000
4010d4: 00 00 00
4010d7: 8b 45 0c    mov     0xc(%ebp),%eax
4010da: 03 45 08    add     0x8(%ebp),%eax
4010dd: 89 45 fc    mov     %eax,-0x4(%ebp)
4010e0: 8b 45 fc    mov     -0x4(%ebp),%eax
4010e3: c9         leave  %eax
4010e4: c3         ret
4010e5: 90         nop
4010e6: 90         nop
4010e7: 90         nop

004010e8 <_printf>:
4010e8: 55          push    %ebp
4010e9: 89 e5       mov     %esp,%ebp
4010eb: 81 ec 28 04 00 00 sub     $0x428,%esp
4010f1: 8d 45 0c    lea     0xc(%ebp),%eax
```

(2) 绘制 showStack() 程序的内存布局，标出[0, 8M)内存空间中，代码段、数据段、堆栈段……各个逻辑段的具体位置，标出本题列出的所有符号的位置。

(采用未修改时的 showStack 程序)



5. (1) 在 Windows 系统中打开 cmd 窗口，重新编译 showStack.c，依然使用我们包里 MinGW/bin 中的 gcc。观察问题 4 中程序的输出结果。解释你观察到的现象。

```
D:\desktop\OS\UNIX V6++V1\oos\src\program>gcc -o showStack.exe showStack.c

D:\desktop\OS\UNIX V6++V1\oos\src\program>showStack
Address of function main: 0x4012f0
Address of function sum: 0x4013db
Address of function printf: 0x401970
Address of global 'version': 0x402000
Address of local 'a': 0x60ff24
Address of local 'b': 0x60ff20
Address of local 'result': 0x60ff1c
result=3
```

如图，在 Windows 中 cmd 重新编译 showStack.c 并运行，发现各个局部变量、函数的地址都与在 Unix V6++中运行的不同了。原因我认为有二。其一是 Windows 操作系统中虚拟内存的结构分布与 UnixV6++的不同。局部变量的地址也不同，说明 Windows 系统的栈底与 Unix V6++的也不一样。其二，全局变量 version 的地址为 0x402000，是因为在 Windows 中.data 段的起始地址为 0x402000。另外，main 函数的入口不再是 0x401000，原因是 Windows 系统在.text 最开头放了 C 运行库的启动代码和辅助函数 (<__gnu_exception_handler> 等)，main 函数的代码放在这些代码后面。

```

00401000 <__gnu_exception_handler@4>:
401000:      55                push    %ebp
401001:      89 e5             mov     %esp,%ebp
401003:      83 ec 18          sub     $0x18,%esp
401006:      89 5d f8          mov     %ebx,0x8(%ebp)

```

```

004012f0 <_main>:
4012f0:      55                push    %ebp
4012f1:      89 e5             mov     %esp,%ebp
4012f3:      83 ec 18          sub     $0x18,%esp
4012f6:      83 c4 f0          and     $0xfffffffff0,%esp

```

(2*) 再次修改 showStack 程序，启用堆，制造 OOM 错误。

```

#include <stdio.h>

#include <stdlib.h> // 必须包含这个头文件才能使用 malloc

void main1() {
    // 每次申请 1MB 的内存
    const int ALLOC_SIZE = 1024 * 1024;

    // 记录总共成功申请了多少 MB
    int total_allocated_mb = 0;

    // 指针，用来接收 malloc 返回的地址
    char *p_mem;

    printf("Starting Out-Of-Memory (OOM) test...\n");

    while (1) {
        p_mem = (char *)malloc(ALLOC_SIZE);

        if (p_mem == 0) {
            // 如果返回 NULL，说明内存耗尽了！
            printf("\n--- OOM ERROR! ---\n");

            printf("malloc failed to allocate more memory.\n");

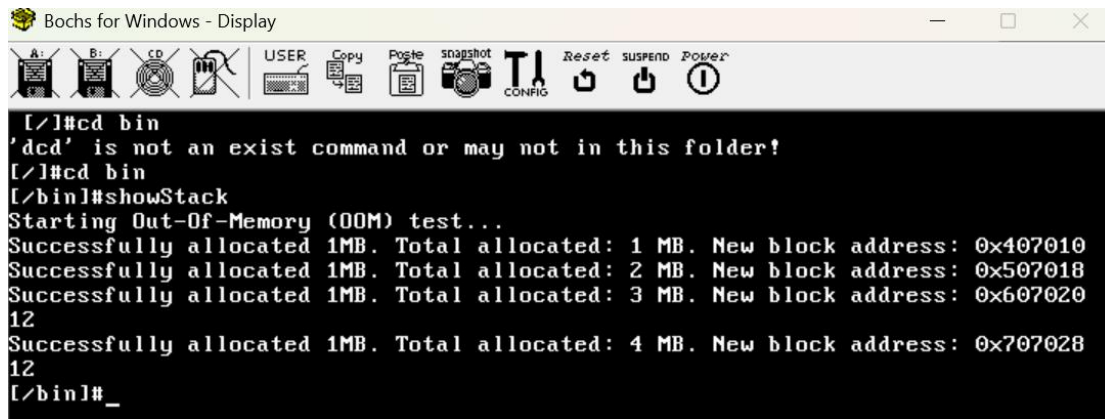
            printf("Total memory successfully allocated before failure: %d MB\n", total_allocated_mb);

            // 制造 OOM 后，程序通常无法继续，这里直接退出循环
            break;
        } else {
            // 如果成功，打印信息并更新计数
            total_allocated_mb++;

            printf("Successfully allocated 1MB. Total allocated: %d MB. New block address: 0x%x\n",
                    total_allocated_mb, p_mem);
        }
    }

    printf("\nOOM test finished.\n");
}

```



```
[/]#cd bin
'dcd' is not an exist command or may not in this folder!
[/]#cd bin
[/bin]#showStack
Starting Out-Of-Memory (OOM) test...
Successfully allocated 1MB. Total allocated: 1 MB. New block address: 0x407010
Successfully allocated 1MB. Total allocated: 2 MB. New block address: 0x507018
Successfully allocated 1MB. Total allocated: 3 MB. New block address: 0x607020
12
Successfully allocated 1MB. Total allocated: 4 MB. New block address: 0x707028
12
[/bin]#_
```

前 4 次 malloc 调用都成功了。堆内存不断向上增长，占用了 4MB 的空间。
程序进入第 5 次循环，再次调用 malloc(1MB)，发生 OOM，程序直接结束，没有预想中的返回空指针，应该是 UnixV6++ 的简化处理。

6. 发现与思考。

我了解到一个可执行文件 (.exe) 远非一个整体，而是被精心划分为 .text (代码)、.data (已初始化数据)、.bss (未初始化数据)、.rdata (只读数据) 等多个功能段。objdump -h 的输出让我明白，编译器和链接器为了效率和安全，会将不同性质的数据分门别类地存放。

objdump 展示的 VMA (虚拟内存地址) 并非随机，而是遵循着严格的对齐规则和布局约定。这让我认识到，可执行文件本身就携带了一份“内存加载蓝图”，指导着操作系统如何正确地将其安置在虚拟内存空间中。

在修改代码重新编译的过程中，我也思考了使用 eslipse 的 build Project 和命令行输入 make clean 和 make all 的区别。经过查阅资料，我发现，这两者最终都依赖于 make 工具，但是 Build Project 执行的是增量构建，只会重新编译那些比目标文件更新的源文件。在命令行界面输入 make clean 和 make all 会从头开始，重新编译项目中的每一个源文件。